

Simulating Single-Particle Trajectories in a Double-Slit Setup

Ion Vlad

contact@ionvlad.com

v1.1.1 December 2025 (v1.5; PDF reformatted June 2026)

Copyright and License

© 2025 Ion Vlad. This work is licensed under the **Creative Commons Attribution 4.0 International License (CC BY 4.0)**.

Attribution: Anyone using information from this work must provide appropriate credit. Please cite as:

Vlad, I. (2025). Simulating Single-Particle Trajectories in a Double-Slit Setup.

Preprint link: <https://zenodo.org/records/17922056>

License Link: <https://creativecommons.org/licenses/by/4.0/>

Source: <https://drive.proton.me/urls/XG5ZKT94A8#ijnqZsisCgjr>

Abstract

The double-slit experiment performed with individual particles has been reproduced many times, yet a universally accepted description of how a single particle can exhibit wave-like behaviour remains elusive. In quantum mechanics we learn that any attempt to detect, observe, or measure a particle inevitably perturbs its trajectory, leading to conclusions that may not reflect the particle's intrinsic dynamics. To explore the motion of a solitary particle without invoking measurement, we wrote a simple program that simulates its trajectory from a source ("the gun") to a detection screen, allowing the particles to interact elastically with the interior surfaces of the slits. By systematically varying wall thickness, slit height, inter-slit gap, particle speed, size, and incident angle, we generate a series of impact distributions ranging from uniform spreads to highly structured multi-cluster patterns. Notably, when the wall thickness allows the particle to bounce several times inside the slit, the emergent patterns evolve into configurations that closely resemble the classic interference fringes of quantum double-slit experiments. Our analysis reveals that these wave-like imprints arise from the superposition of numerous classical trajectories produced by successive bounces off the slit walls, rather than from an intrinsic wave property of the particles. Consequently, the study suggests that apparent interference phenomena can be reproduced within a purely classical framework, highlighting the pivotal role of mechanical interactions between particles and slit boundaries in shaping observed detection patterns.

Introduction

While the particles are emitted at various angles, some will pass through the slit and leave a red imprint on the screen. The particles that pass through the upper slit remain blue, whereas those that pass through the lower slit turn red. This method records which slit each particle passes through without any physical interaction with the particles (or detectors).

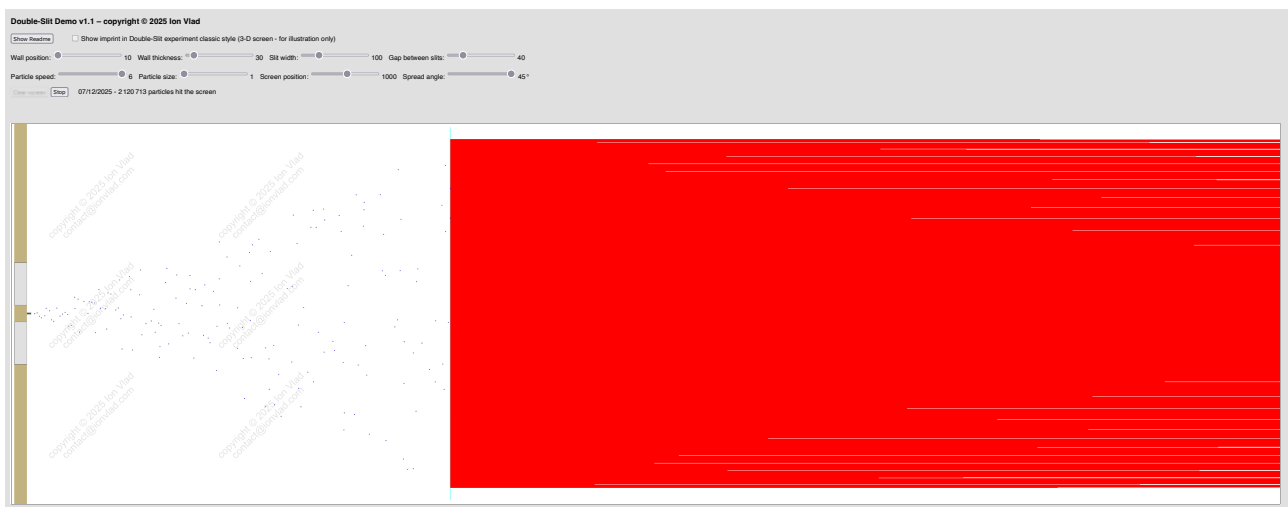
The screen can be displayed in either 2-D or 3-D. However, the 2-D view records each impact individually, making the accumulation of dots far more reliable for detailed analysis. The 3-D mode is provided for illustrative purposes only.

Because it's a computer simulation, we used pixels as the measurement unit, but any unit works as long as the proportions are preserved. Also, because the particle's mass is assumed to be negligible, the simulation treats each bounce off the wall or slit as perfectly elastic—so the particle's speed remains unchanged after every collision.

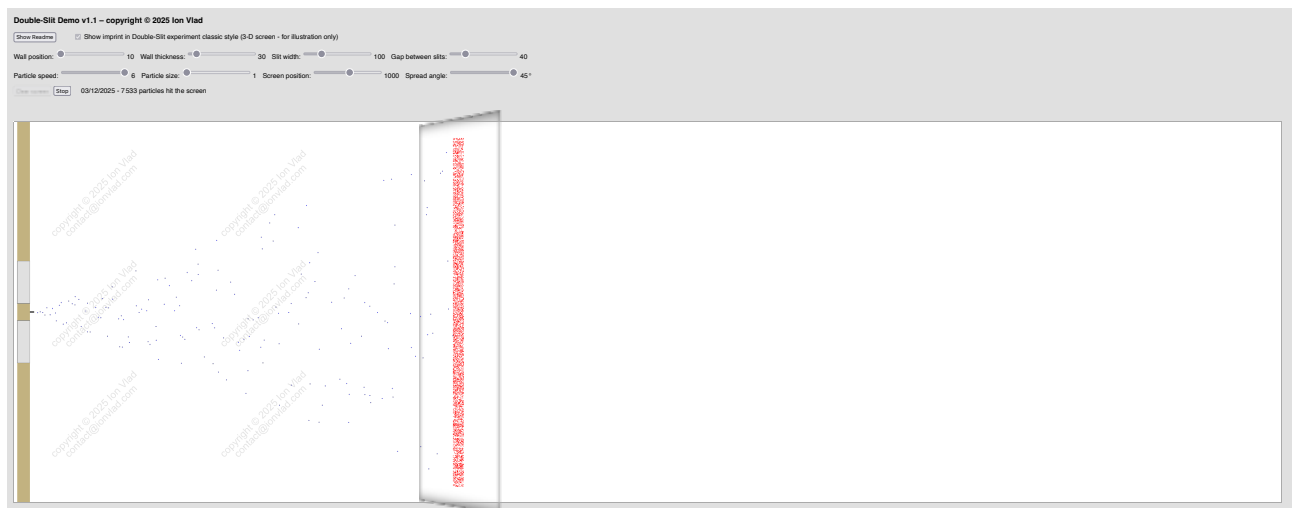
The results are illustrated in the screenshots below.

1. No Obstacle Between Source and Screen

When there is no wall separating the gun from the screen, the impact pattern on the detector is essentially uniform—a flat spread of hits. This can be seen on Screenshot 1: 2D and Screenshot 1: 3D.



Screenshot 1: 2D



Screenshot 1: 3D

2. Introducing a Wall with Two Slits

Adding a barrier that contains two narrow openings produces a markedly different pattern. The screen now shows two distinct clusters of impacts, corresponding to particles that passed through each slit (Screenshot 2: 2D and Screenshot 2: 2D Intermediate).



Screenshot 2: 2D



Screenshot 2: 2D Intermediate

3. Varying Wall Thickness, Slit Height, and Gap

Increasing the wall's thickness, adjusting the slit height, or widening the separation between the slits yields a more intricate distribution. In Screenshot 3: 2D and Screenshot 3: 2D Intermediate, the imprint resolves into four separate bundles, even though only two slits are present. This occurs because particles can bounce off any of the four interior surfaces of the slits.



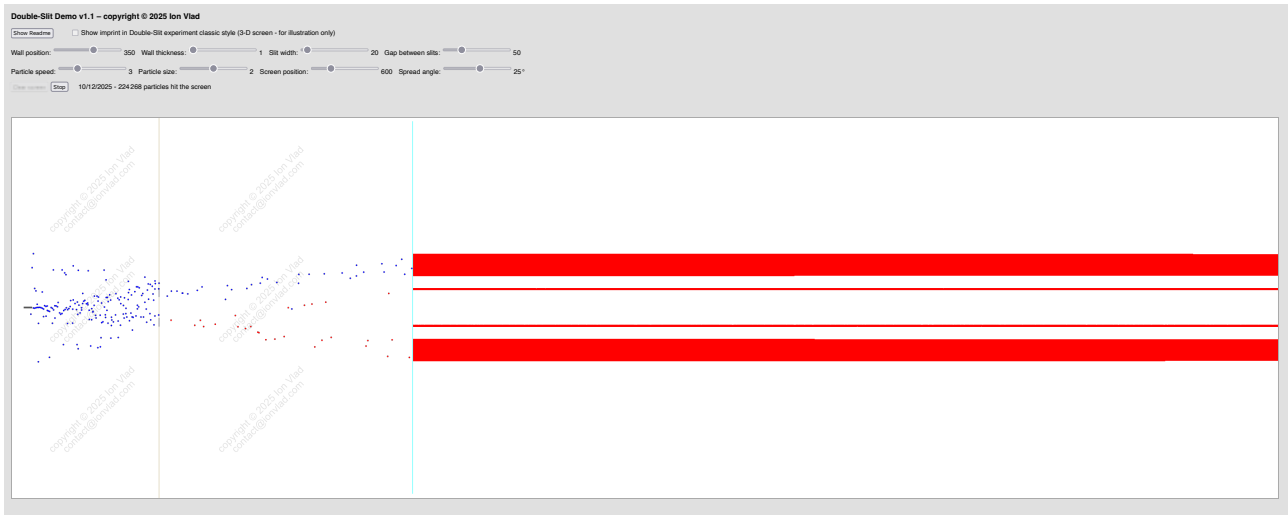
Screenshot 3: 2D



Screenshot 3: 2D Intermediate

4. Thin Walls Relative to Particle Size

Surprisingly, the same multi-bundle pattern emerges when the wall's thickness is reduced to less than the particle's diameter, as shown in Screenshot 4.2: 2D and in Screenshot 4.2: 2D Intermediate.



Screenshot 4.2: 2D



Screenshot 4.2: 2D Intermediate

5. Further Parameter Tuning

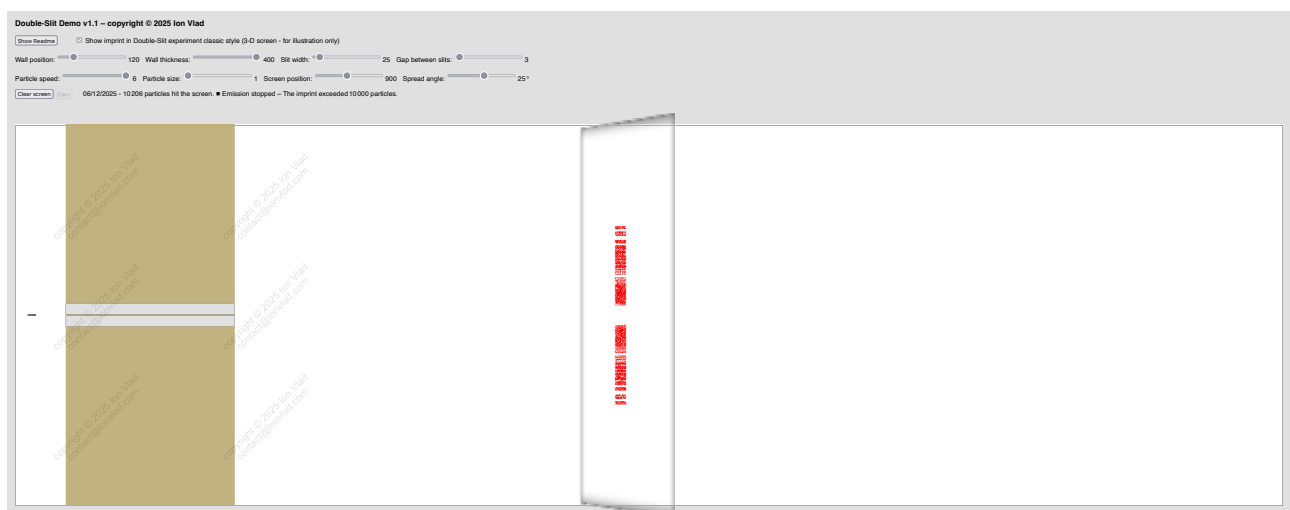
By fine-tuning additional parameters—such as particle size, speed, incident angle, slit dimensions, and the distances between gun, wall, and screen—the impact pattern becomes increasingly complex. Screenshot 5: 2D displays an even larger number of distinct bundles.



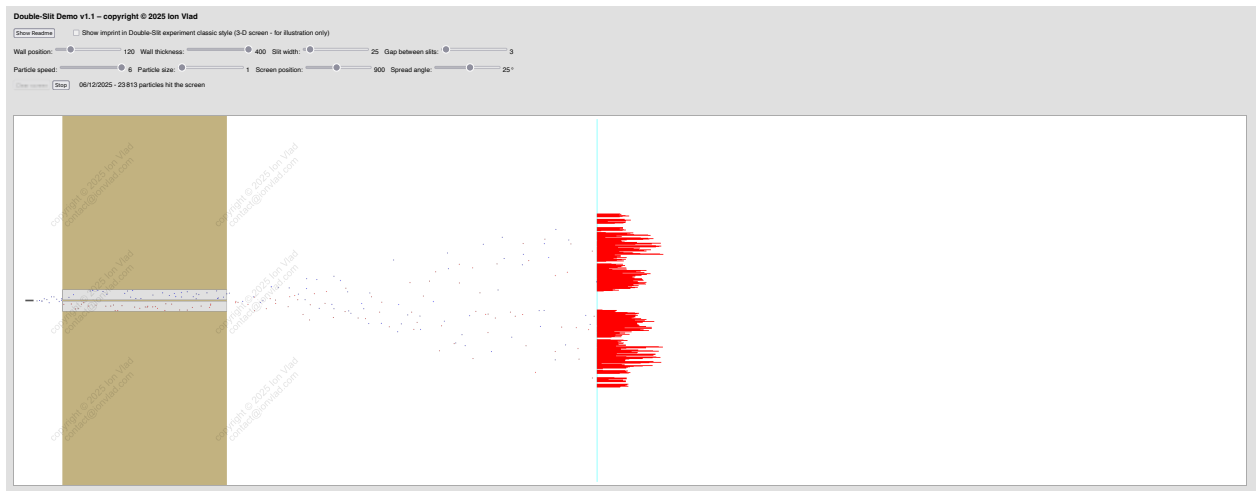
Screenshot 5: 2D

6. Emergence of a Wave-Like Distribution

When the system is configured appropriately, the resulting imprint begins to resemble a classic interference pattern, with alternating high- and low-intensity regions reminiscent of a wave. This effect is evident in Screenshot 6: 3D and Screenshot 6: 2D Intermediate.



Screenshot 6: 3D



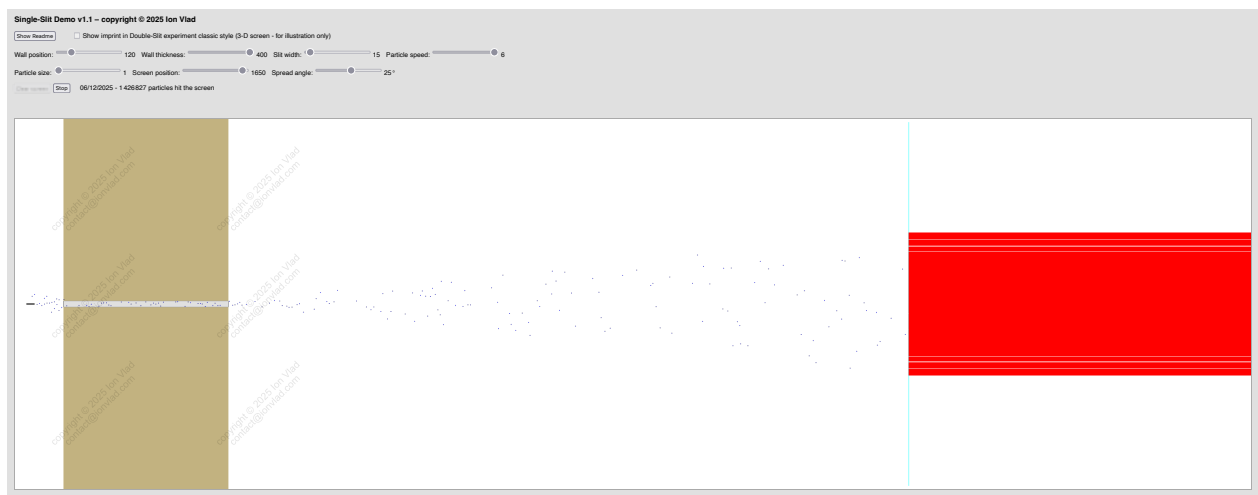
Screenshot 6: 2D Intermediate

7. Single slit wall

Surprisingly, the same multi-bundle pattern emerges even when the wall has only a single slit, as shown in Screenshot 7: 3D and in Screenshot 7: 2D.



Screenshot 7: 3D



Screenshot 7: 2D

Discussion

The observed “wave-like” imprint appears to arise from the particle’s propensity to bounce off the interior surfaces of the slits. Each collision can alter the particle’s direction while conserving its speed, allowing it to emerge from the slit at a variety of angles. Consequently, the final distribution on the screen reflects a superposition of many possible trajectories rather than a true wave phenomenon.

From this perspective, any detector, beam splitter, or deflector that interacts with the particle inevitably modifies its path, potentially leading to misleading interpretations of the underlying physics.

Conclusion

The simulations suggest that the interference-like patterns commonly attributed to wave behaviour can be reproduced by classical bouncing dynamics alone. In other words, the “wave-like” imprint may stem from the mechanical interactions of a single particle with the slit walls, rather than from an intrinsic wave nature of the particle itself. Reproducing the pattern does not falsify quantum mechanics, but highlights the importance of boundary interactions. While a single particle doesn’t exhibit wave-like behaviour, a collection of particles can. Conversely, a wave can be described in terms of particle behaviour—such as a photon in light or a water molecule in a ripple.

As we can see, the “wave-like” imprint occurs when the thickness of the wall is far greater than the particle diameter, which in previous experiments

was typically the default configuration since the chosen particle was an electron.

If we are to reproduce this experiment with macro particles, in order to get the same results, we have to maintain geometric similarity (particle diameter - wall thickness - particle mass - speed). However, there are some limitations since a greater mass would cause the particle to lose momentum inside the slit.

Appendix A contains the source code used to generate all results presented in this paper. For convenience, the latest maintained version of the software and additional screenshots are also available at the GitHub repository and Zenodo archive.”

<https://github.com/Ion-Vlad/Simulating-Single-Particle-Trajectories-in-a-Double-Slit-Setup>

Vlad, I. (2025). *Simulation of single-particle trajectories in a double-slit geometry* (Version 1.1.1) [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.17922056>

DOI [10.5281/zenodo.17922056](https://doi.org/10.5281/zenodo.17922056)

Appendix A – Double_slit_simulation_v1.1.html

Source Code

```

1  <!------- LICENSE ----->
2  <!-- Copyright © 2025 Ion Vlad -->
3  <!-- This program is released under the Creative Commons Attribution 4.0 International License -->
4  <!-- (CC BY 4.0). -->
5  <!-- When you modify or redistribute the code you must: -->
6  <!--   • Keep this copyright notice intact. -->
7  <!--   • Add a line "tweaked by 'your name' " (or similar) after the notice. -->
8  <!--   • If you rewrite the program in another language, keep the above notice visible in the -->
9  <!--     UI/screenshots and include a link to the original repository: -->
10 <!-- https://github.com/Ion-Vlad/Simulating-Single-Particle-Trajectories-in-a-Double-Slit-Setup -->
11 <!-- or/and: -->
12 <!-- https://drive.proton.me/urls/XG5ZKT94A8#ijnqZsisCgjjv -->
13 <!-- -->
14 <!-- Full license text: https://creativecommons.org/licenses/by/4.0/ -->
15 <!-- -->
16 <!-- Thank you for understanding! -->
17 <!------->
18
19 <!DOCTYPE html>
20 <html lang='en'>
21 <head>
22   <meta charset='UTF-8'>
23   <title>Double-Slit Demo</title>
24   <style>
25     body {
26       font-family:sans-serif;
27       margin:20px;
28       background:#E0E0E0;
29       align-items:center;
30       height:100vh;
31     }
32     .scene {
33       perspective: 8000px; /* camera distance */
34       perspective-origin: left center;
35       position: relative;
36       width: 3050px;
37       height: 1020px;
38       transform-style: preserve-3d;
39       /* border: 1px solid #aaa; */
40     }
41     #controls {margin-bottom:12px;}
42     canvas {position:absolute;top:0;left:0;display:block;}
43     #bgCanvas {z-index:1;}
44     #wmCanvas {z-index:2;}
45     /* Main canvas – the thing you normally draw on */
46     #fgCanvas {
47       z-index:10;
48       border: 1px solid #aaa;
49       cursor: crosshair;
50     }
51     /* The Classic style canvas */
52     #csCanvas {
53       z-index:3;
54       width: 600px;
55       height: 880px;
56       background: transparent; /* explicit – default is already transparent */
57       border:2px solid #888;
58       position:absolute;
59       pointer-events: none; /* let mouse events fall through the canvas */
60       top:0; /* align top edges */
61       left:400px; /* start at the right edge of the big canvas */
62       transform-origin: left center; /* pivot around its left edge */
63       transform: rotateY(-80deg); /* tilt it toward the viewer */
64       box-shadow: inset 0 0 30px rgba(0,0,0,.6);
65       -webkit-print-color-adjust: exact;
66       print-color-adjust: exact; /* older spec, some browsers still recognise it */
67       color-adjust: exact;
68       display:none;
69     }
70   </style>

```

```

71  /* The magnifier "lens" – positioned absolutely over the page */
72  #lensCanvas {
73      z-index:999;
74      position:absolute;
75      pointer-events:none;          /* let mouse events fall through main canvas */
76      border:2px solid #333;
77      border-radius: 50%;
78      box-shadow:0 0 6px rgba(0,0,0,.4);
79      background:#fff;
80      display:none;                /* hidden until we actually hover */
81  }
82  label {margin-right:10px;}
83  /* Adds 20 px space to the right of the button */
84  .button {margin-right:20px;}
85  /* Only a tiny bit of style for readme so the box is readable */
86  #readme{
87      display:block;
88      margin-top:0.5rem;
89      padding:0.5rem;
90      border:1px solid #999;
91      background:#fefefe;
92  }
93  button:disabled {
94      opacity: 0.5;
95      cursor: not-allowed;
96  }
97  .spacer {
98      display: inline-block;
99      width: 2rem;    /* control the amount of space between Hide Readme button and checkbox */
100  }
101  </style>
102  </head>
103  <body>
104      <h3>Double-Slit Demo v1.1 – copyright © 2025 Ion Vlad</h3>
105      <div id='readme' style='display:block'>
106          <strong>Disclaimer:</strong><br>
107          The program is built with HTML + JavaScript, allowing anyone to read the code and run it on
108          most desktop browsers or Android devices without needing additional libraries or environments. For
109          obvious reasons it works in Safari on desktop, but not on iPhone or iPad – I haven't tested it on
110          an Android tablet.
111          Because this isn't a programming-project assignment, I wanted to keep the code as short and
112          simple as possible, avoiding unnecessary graphics or functions that would be required to make it
113          run on iPhone or iPad. I have tested it on an Android phone with Chrome, on Microsoft Edge and
114          Opera on a PC, on Safari, Firefox and Opera on a MacBook Pro, and on Linux.
115          Since this program is meant solely for demonstration purposes and isn't intended for in-depth
116          research, avoid running the simulation for long periods on low-memory or slow computers.
117          I'm not a professional software engineer, so please refrain from critiquing my programming
118          skills—the simulation's concept is what matters most. However, I'm open to any constructive
119          suggestions.
120          <p></p>
121          All files and documents related to this work (including additional screenshots) can be found
122          here:
123          <p></p>
124          <a href='https://github.com/Ion-Vlad/Simulating-Single-Particle-Trajectories-in-a-Double-Slit-Setup'>https://github.com/Ion-Vlad/Simulating-Single-Particle-Trajectories-in-a-Double-Slit-Setup</a>
125          <p></p>
126          <a href='https://drive.proton.me/urls/XG5ZKT94A8#ijnqZsisCgjjv'>https://drive.proton.me/urls/XG5ZKT94A8#ijnqZsisCgjjv</a>
127          <p></p>
128          Vlad, I. (2025). *Simulation of single-particle trajectories in a double-slit geometry*
129          (Version 1.1) [Computer software]. Zenodo. <a href='https://doi.org/10.5281/zenodo.17922056'>https://doi.org/10.5281/zenodo.17922056</a>
130          <p></p>
131          1. This program does <strong>not</strong> simulate how photons or waves behave; it merely
132          demonstrates that particles can leave a "wave-like" imprint on the screen even though they
133          <strong>do not</strong> behave like waves. <strong><u>Reproducing the pattern does not falsify
134          quantum mechanics, but highlights the importance of boundary interactions.</u></strong>
135          <p></p>
136          2. Because the particles are emitted randomly, two simulations with identical parameters may
137          produce slightly different imprints.
138          <p></p>
139          3. Collisions between particles are not shown, as particles can be fired one at a time.
140          <p></p>
141          4. At high speeds (> 4), a small particle can pass through a thin wall. This is normal
142          single-particle behaviour and should not be interpreted as a programming bug.
143          <p></p>
144          5. The interface is self-explanatory: adjust the settings and press <strong>Start</strong>.
145          Some particles will pass through the slit and leave a red imprint on the screen. The particles
146          that pass through the upper slit remain blue, whereas those that pass through the lower slit turn
147          red. This method records which slit each particle passes through without any physical interaction
148          with the particles (or detectors).
149          <p></p>

```

```

150     6. When enough particles have struck the screen, press <strong>Stop</strong> and analyse the
151     imprint. You can press <strong>Start</strong> again if you want the imprint to “build up” further.
152     <p></p>
153     7. The <strong>Clear Screen</strong> button removes all red imprints and prepares the program
154     for a new simulation, but it does not alter the current settings.
155     <p></p>
156     8. During the simulation the sliders (controls) are disabled, and they become re-enabled when
157     the Clear Screen button is clicked.
158     <p></p>
159     9. If the <strong>Screen Position</strong> setting is too high and the screen moves out of
160     view, while the simulation is running (emitting), adjust the browser’s zoom level (or change the
161     screen resolution) until the entire canvas fits within the viewport. Saving screenshots works best
162     in Firefox by selecting Print → Save as PDF.
163     <p></p>
164     10. Hover the mouse over the screen line to display a magnifier. This lets you inspect the
165     imprint at the pixel level.
166     <p></p>
167     11. The sliders can be adjusted incrementally with the arrow keys on the keyboard.
168     <p></p>
169     12. In the 3-D view, the number of visible dots can be fewer than the total number of
170     particles that hit the screen. This occurs when multiple particles land at exactly the same
171     coordinates; they overlap and appear as a single dot. In contrast, the 2-D view records every
172     impact individually, so <strong>the accumulation of dots is more reliable for detailed analysis.
173     </strong> In 3-D view mode the simulation runs considerably slower, so we cap it at 10 000
174     particles.
175     <p></p>
176     13. <strong>Browser-specific issues:</strong>
177     a. Safari – Captures the 3-D view correctly, but saves it as a flat rectangle, which can
178     lead to misalignment in the imprint.
179     b. Opera, Edge and Chrome – The 3-D view is captured accurately, yet the resulting image
180     appears darker than the actual screen. Also Chrome is running slow.
181     c. Firefox – Saves the screenshot properly, but refreshing the page causes a strange
182     misalignment and the sliders fail to update; a full page reload resolves the problem.
183     Note: The behaviour may vary depending on the computer hardware and browser version. It’s
184     advisable to reload the page after each simulation.
185     <p></p>
186     14. Do not run multiple simulations simultaneously in the same browser. Launch the program in
187     full-screen mode and prevent the computer from going to sleep.
188     </div>
189     <p></p>
190     <button id='readmeBtn'>Hide Readme</button>
191     <span class='spacer'></span>
192     <input type='checkbox' id='iClassic' checked>
193     Show imprint in Double-Slit experiment classic style (3-D screen - for illustrative purposes
194     only)
195     <p></p>
196     <div id='controls'>
197     <!-- sliders -->
198     <label>Wall position:
199     <input type='range' id='wallPosSlider' min='10' max='575' value='300'>
200     <span id='wallPosVal'>300</span>
201     </label>
202     <label>Wall thickness:
203     <input type='range' id='thickSlider' min='1' max='400' value='30'>
204     <span id='thickVal'>30</span>
205     </label>
206     <label>Slit width:
207     <input type='range' id='slitSizeSlider' min='3' max='420' value='100'>
208     <span id='slitSizeVal'>100</span>
209     </label>
210     <label>Gap between slits:
211     <input type="range" id="slitGapSlider" min="1" max="200" value="40">
212     <span id="slitGapVal">40</span>
213     </label>
214     <p></p>
215     <label>Particle speed:
216     <input type='range' id='particleSpeedSlider' min='2' max='6' value='6'>
217     <span id='particleSpeedVal'>6</span>
218     </label>
219     <label>Particle size:
220     <input type='range' id='particleSlider' min='1' max='3' value='1'>
221     <span id='particleVal'>1</span>
222     </label>
223     <label>Screen position:
224     <input type='range' id='screenPosSlider' min='200' max='1700' value='600'>
225     <span id='screenPosVal'>600</span>
226     </label>
227     <label>Spread angle:
228     <input type='range' id='angleSlider' min='1' max='45' value='45'>

```

```

229     <span id='angleVal'>45</span> °
230 </label>
231 </p></p>
232 <button id='resetBtn'>Clear screen</button>
233 <button class='button' id='toggleBtn'>Start</button>
234 <span id='date'></span>
235 <span id='info'></span>
236 </div>
237 <div class='scene' style='overflow:hidden;'>
238     <canvas id='csCanvas' width='600' height='880'></canvas>
239     <canvas id='bgCanvas' width='3000' height='900'></canvas>
240     <canvas id='fgCanvas' width='3000' height='900'></canvas>
241     <canvas id='wmCanvas' width='1200' height='900'></canvas>
242     <canvas id='lensCanvas' width='150' height='150'></canvas>
243 </div>
244 <script>
245     // Get current date
246     let now = new Date();
247     let date = now.toLocaleDateString();
248     // Insert date and time into HTML
249     document.getElementById('date').innerHTML = date + ' - ';
250     // ----- GLOBAL STATE -----
251     let emitting = false; // true while particles should be generated
252     let hitLimitReached = false; // true once we have hit the limit for 3D mode
253     // ----- Global canvas references -----
254     const bgCanvas = document.getElementById('bgCanvas');
255     const fgCanvas = document.getElementById('fgCanvas');
256     const wmCanvas = document.getElementById('wmCanvas');
257     const csCanvas = document.getElementById('csCanvas');
258     const bg = bgCanvas.getContext('2d');
259     const fg = fgCanvas.getContext('2d');
260     const wm = wmCanvas.getContext('2d');
261     const cs = csCanvas.getContext('2d');
262     const W = bgCanvas.width;
263     const H = bgCanvas.height;
264     const canvasTop = 50;
265     const screenTop = 10; // make the screen a bit smaller than the canvas
266     const screenBottom = H - 10;
267     const gunX = 50; // fixed gun x-position
268     const gunY = H / 2; // gun y-position (centre)
269     // ----- Watermark style and settings -----
270     wm.globalAlpha = 0.12; // transparency
271     wm.font = '24px Arial'; // size & font
272     wm.textAlign = 'center';
273     wm.textBaseline = 'middle';
274     const lineSpacing = 260; // vertical distance between rows
275     const colSpacing = 400; // horizontal distance between columns
276     const rows = Math.ceil(300 / lineSpacing) + 1;
277     const cols = Math.ceil(300 / colSpacing) + 1;
278     const angleWm = -Math.PI / 4; // ~-45° rotation
279     // ----- Set up the lens canvas (the magnifier) -----
280     const lens = document.getElementById('lensCanvas');
281     const ctxLens = lens.getContext('2d');
282     const lensSize = lens.width;
283     const zoomFactor = 5; // how much bigger the view appears
284     // radius of the source area we'll copy from the main canvas
285     const srcRadius = lensSize / (2 * zoomFactor);
286     // ----- Simulation state (will be updated by UI) -----
287     let slitHeight = 25; // slit height in Classic style
288     let wallX = 300; // left side of the wall (slider)
289     let wallThickness = 30; // wall thickness (slider)
290     let slitWidth = 100; // width of each slit (slider)
291     let slitGap = 40; // vertical distance between the two slits (slider)
292     let screenPos = 600; // distance from wall to screen (slider)
293     let spreadAngle = 45; // spread angle at the gun muzzle (slider)
294     let particleSize = 1; // particle size (slider)
295     let particleSpeed = 6; // particle speed (slider)
296     let screenX = wallX + wallThickness + screenPos; // absolute screen x-coord
297     // ----- Hit counters -----
298     const hitCount = new Uint32Array(H); // how many particles landed at each y-row
299     let totalHits = 0; // total hits for info
300     const hitOffsets = []; // Parallel to hitCount – each row holds an array
301     // ----- UI elements -----
302     const chkClassic = document.getElementById('iClassic');
303     const wallPosSlider = document.getElementById('wallPosSlider');
304     const wallPosVal = document.getElementById('wallPosVal');
305     const thickSlider = document.getElementById('thickSlider');
306     const thickVal = document.getElementById('thickVal');
307     const screenPosSlider = document.getElementById('screenPosSlider');

```

```

308     const screenPosVal = document.getElementById('screenPosVal');
309     const angleSlider = document.getElementById('angleSlider');
310     const angleVal = document.getElementById('angleVal');
311     const slitSizeSlider = document.getElementById('slitSizeSlider');
312     const slitSizeVal = document.getElementById('slitSizeVal');
313     const particleSpeedSlider = document.getElementById('particleSpeedSlider');
314     const particleSpeedVal = document.getElementById('particleSpeedVal');
315     const particleSlider = document.getElementById('particleSlider');
316     const particleVal = document.getElementById('particleVal');
317     const toggleBtn = document.getElementById('toggleBtn');
318     const resetBtn = document.getElementById('resetBtn');
319     const readmeBtn = document.getElementById('readmeBtn');
320     const info = document.getElementById('info');
321     // Move the canvases down inside the scene
322     fgCanvas.style.top = canvasTop + 'px';
323     bgCanvas.style.top = canvasTop + 'px';
324     wmCanvas.style.top = canvasTop * 2 + 'px';
325     csCanvas.style.display = 'block';
326     // ---- Helper to keep screenX in sync whenever wallX, wallThickness or screenPos change
327     function updateScreenX() {
328         screenX = wallX + wallThickness + screenPos;
329     }
330     // ---- Slider callbacks ----
331     chkClassic.addEventListener('change', () => {
332         const disabled = chkClassic.checked; // true → disabled
333         csCanvas.style.display = chkClassic.checked ? 'block' : 'none'; //show the Classic
334 style screen
335         drawBackground();
336     });
337     function isSafari() { // check if the browser is Safari
338         const ua = navigator.userAgent;
339         // Safari's UA contains "Safari" but NOT "Chrome"/"Chromium"/"CriOS"
340         return /^(?!chrome|android).*safari/i.test(ua);
341     }
342     if (isSafari) { // Safari doesn't focus on sliders, so they can't be ajusted with
343 keyboard... we fix it now...
344         wallPosSlider.addEventListener('click', () => {
345             wallPosSlider.focus();
346         });
347         thickSlider.addEventListener('click', () => {
348             thickSlider.focus();
349         });
350         screenPosSlider.addEventListener('click', () => {
351             screenPosSlider.focus();
352         });
353         angleSlider.addEventListener('click', () => {
354             angleSlider.focus();
355         });
356         slitSizeSlider.addEventListener('click', () => {
357             slitSizeSlider.focus();
358         });
359         slitGapSlider.addEventListener('click', () => {
360             slitGapSlider.focus();
361         });
362         particleSpeedSlider.addEventListener('click', () => {
363             particleSpeedSlider.focus();
364         });
365         particleSlider.addEventListener('click', () => {
366             particleSlider.focus();
367         });
368     }
369     wallPosSlider.addEventListener('input', () => {
370         wallX = Number(wallPosSlider.value);
371         wallPosVal.textContent = wallX;
372         updateScreenX();
373         drawBackground();
374     });
375     thickSlider.addEventListener('input', () => {
376         wallThickness = Number(thickSlider.value);
377         thickVal.textContent = wallThickness;
378         updateScreenX();
379         drawBackground();
380     });
381     screenPosSlider.addEventListener('input', () => {
382         screenPos = Number(screenPosSlider.value);
383         screenPosVal.textContent = screenPos;
384         updateScreenX();
385         drawBackground();
386     });

```

```

387 angleSlider.addEventListener('input', () => {
388     spreadAngle = Number(angleSlider.value);
389     angleVal.textContent = spreadAngle;
390     drawBackground();
391 });
392 slitSizeSlider.addEventListener('input', () => {
393     slitWidth = Number(slitSizeSlider.value);
394     slitSizeVal.textContent = slitWidth;
395     drawBackground();
396 });
397 slitGapSlider.addEventListener('input', () => {
398     slitGap = Number(slitGapSlider.value);
399     slitGapVal.textContent = slitGap;
400     drawBackground();
401 });
402 particleSpeedSlider.addEventListener('input', () => {
403     particleSpeed = Number(particleSpeedSlider.value);
404     particleSpeedVal.textContent = particleSpeed;
405     drawBackground();
406 });
407 particleSlider.addEventListener('input', () => {
408     particleSize = Number(particleSlider.value);
409     particleVal.textContent = particleSize;
410     drawBackground();
411 });
412 // ----- Toggle the visibility of the #readme div -----
413 document.getElementById('readmeBtn').addEventListener('click', function () {
414     var box = document.getElementById('readme');
415     if (box.style.display === 'none' || box.style.display === '') {
416         box.style.display = 'block';
417         this.textContent = 'Hide Readme';
418     } else {
419         box.style.display = 'none';
420         this.textContent = 'Show Readme';
421     }
422 });
423 // ----- Reset button -----
424 resetBtn.addEventListener('click', () => {
425     hitOffsets.length = 0; //clear the array
426     hitCount.fill(0);
427     totalHits = 0;
428     bg.clearRect(0, 0, W, H); // clear canvas
429     fg.clearRect(0, 0, W, H); // clear canvas
430     drawBackground();
431     ctxLens.clearRect(0, 0, lensSize, lensSize); // clear magnifier
432     hitLimitReached = false; // we can emit again
433     toggleBtn.disabled = false; // re-enable the Start/Stop button
434     toggleBtn.textContent = 'Start';
435     emitting = false; // make sure we aren't still emitting
436     controlsEnabled(); // Activate controls only after the screen has been cleared
437     // Get current date
438     let now = new Date();
439     let date = now.toLocaleDateString();
440     // Insert date and time into HTML
441     document.getElementById('date').innerHTML = date + ' - ';
442     info.textContent = ' ';
443 });
444 // ----- Start / Stop logic -----
445 toggleBtn.addEventListener('click', () => {
446     emitting = !emitting;
447     toggleBtn.textContent = emitting ? 'Stop' : 'Start';
448     controlsDisabled();
449 });
450 function controlsEnabled() {
451     chkClassic.disabled = false;
452     wallPosSlider.disabled = false;
453     thickSlider.disabled = false;
454     slitSizeSlider.disabled = false;
455     slitGapSlider.disabled = false;
456     particleSpeedSlider.disabled = false;
457     particleSlider.disabled = false;
458     screenPosSlider.disabled = false;
459     angleSlider.disabled = false;
460 }
461 function controlsDisabled() {
462     chkClassic.disabled = true;
463     resetBtn.disabled = emitting; // controled directly from Start button
464     wallPosSlider.disabled = true;
465     thickSlider.disabled = true;

```

```

466     slitSizeSlider.disabled = true;
467     slitGapSlider.disabled = true;
468     particleSpeedSlider.disabled = true;
469     particleSlider.disabled = true;
470     screenPosSlider.disabled = true;
471     angleSlider.disabled = true;
472 }
473 // ----- Geometry helpers -----
474 // ----- Calculate slit positions -----
475 function getSlitRects() {
476     const total = 2 * slitWidth + slitGap;
477     const startTop = (H - total) / 2;
478     return [
479         { top: startTop, bottom: startTop + slitWidth },
480         { top: startTop + slitWidth + slitGap,
481           bottom: startTop + 2 * slitWidth + slitGap }
482     ];
483 }
484 // ----- Background drawing (static elements) -----
485 function drawBackground() {
486     bg.clearRect(0, 0, W, H);
487     bg.fillStyle = '#fff';
488     bg.fillRect(0, 0, W, H);
489     // gun
490     bg.strokeStyle = '#555';
491     bg.lineWidth = 4;
492     bg.beginPath();
493     bg.moveTo(gunX - 20, gunY);
494     bg.lineTo(gunX, gunY);
495     bg.stroke();
496     // wall
497     bg.fillStyle = '#c2b280';
498     bg.fillRect(wallX, 0, wallThickness, H);
499     // slits (clear rectangles)
500     const slits = getSlitRects();
501     bg.clearRect(wallX, slits[0].top, wallThickness, slitWidth);
502     bg.clearRect(wallX, slits[1].top, wallThickness, slitWidth);
503     bg.strokeStyle = '#000';
504     bg.lineWidth = 0.1;
505     bg.strokeRect(wallX, slits[0].top, wallThickness, slitWidth);
506     bg.strokeRect(wallX, slits[1].top, wallThickness, slitWidth);
507     // Classic style logic
508     if (chkClassic.checked) {
509         const CLASSIC_X_OFFSET = 80; // position 3-D screen with the 2-D line - bottom of
510         the 3D screen is not screenX
511         csCanvas.style.left = screenX - CLASSIC_X_OFFSET + 'px';
512         csCanvas.style.top = screenTop + canvasTop + 'px';
513         // hide screen line
514         bg.strokeStyle = 'white';
515         bg.lineWidth = 1;
516         bg.beginPath();
517         bg.moveTo(screenX, screenTop);
518         bg.lineTo(screenX, screenBottom);
519         bg.stroke();
520     } else {
521         bg.strokeStyle = 'cyan';
522         bg.lineWidth = 1;
523         bg.beginPath();
524         bg.moveTo(screenX, screenTop);
525         bg.lineTo(screenX, screenBottom);
526         bg.stroke();
527     }
528 }
529 // ----- Particle class -----
530 class Particle {
531     constructor() {
532         this.r = particleSize;
533         this.x = gunX;
534         this.y = gunY;
535         // random launch angle of 45° or other chosen value
536         const deg = (Math.random() * spreadAngle) - spreadAngle / 2;
537         const rad = deg * Math.PI / 180;
538         const speed = particleSpeed;
539         this.vx = Math.cos(rad) * speed;
540         this.vy = Math.sin(rad) * speed;
541         this.alive = true;
542         this.color = 'blue'; // default (#0066ff)
543     }
544     update() {

```

```

545     if (!this.alive) return;
546     // move
547     this.x += this.vx;
548     this.y += this.vy;
549     // ----- Collision with wall -----
550     if (this.vx > 0 &&
551         this.x + this.r >= wallX &&
552         this.x - this.r <= wallX + wallThickness) {
553         const intersectY = this.y;
554         const slits = getSlitRects();
555         // helper: is Y inside a given slit?
556         const inside = (y, slit) => y >= slit.top && y <= slit.bottom;
557         const inUpper = inside(intersectY, slits[0]);
558         const inLower = inside(intersectY, slits[1]);
559         if (inUpper || inLower) {
560             // inside a slit – possible bounce on top/bottom edge
561             // Bounce constant: If bounce < "particleSize + particleSize/2"
562             // the bounce inside the slit "misbehave" (the particle sometimes
563             // can be "absorbed" by the wall. If bounce > "particleSize +
564             // particleSize/2" the particle inside the slit can bounce before
565             // touching the slit wall.
566             // CHANGE bounce VALUE IF YOU WANT SOME PARTICLES TO BE ABSORBED BY THE
567 WALL
568             const bounce = particleSize + particleSize / 2;
569             const cur = inUpper ? slits[0] : slits[1];
570             // remember which slit we passed through (colour cue)
571             this.color = inUpper ? 'blue' : 'red'; // upper slit blue and lower slit turn
572 red
573             if (Math.abs(intersectY - cur.top) < bounce) {
574                 this.vy = Math.abs(this.vy); // reflect downwards
575             } else if (Math.abs(intersectY - cur.bottom) < bounce) {
576                 this.vy = -Math.abs(this.vy); // reflect upwards
577             }
578             // otherwise just pass straight through
579         } else {
580             // solid part of the wall → elastic bounce
581             this.vx = -this.vx;
582         }
583     }
584     // ----- Reaching the screen -----
585     if (this.x + this.r >= screenX) {
586         // the particle reached the screen on x axis
587         const iy = Math.max(0, Math.min(H - 1, Math.round(this.y)));
588         if (this.y >= screenTop && this.y <= screenBottom) {
589             // the particle reached the screen on y axis and
590             // didn't escaped at the top or bottom of the screen
591             hitCount[iy]++; // record the hit in array
592             totalHits++; // record the hit for info
593             //--- Classic Style - Compute a random offset for THIS hit -----
594             // Since the particles are emitted at random on horizontal plane
595             // they will have to have the same behaviour on vertical plane, so we have to
596             // create a "mock" random vertical distribution on the 3D screen
597             if (chkClassic.checked){
598                 const randomImprint = Math.random() * slitHeight; // stay inside the screen
599                 // Store it in the per-row array
600                 if (!hitOffsets[iy]) hitOffsets[iy] = [];
601                 hitOffsets[iy].push(randomImprint);
602             }
603             //-----
604             this.alive = false; // stop moving
605             // format number of screen hits as "# ### ###" and display it
606             if (chkClassic.checked && totalHits >= 10000) {
607                 haltEmission(); // in 3-D view the performance is lower
608                 info.textContent = `${totalHits.toLocaleString('fr-FR')} particles hit
609 the screen. ■ Emission stopped – The imprint exceeded 10 000 particles.`;
610             } else {
611                 info.textContent = `${totalHits.toLocaleString('fr-FR')} particles hit
612 the screen`
613             }
614             return;
615         }
616     }
617     // Kill the particles outside the canvas
618     if (this.x < gunX || this.y > H || this.y < 1) this.alive = false; // stop moving
619     // draw moving particle
620     this.draw();
621 }
622 draw() {
623     fg.fillStyle = this.color; // use the per-particle colour

```

```

624         fg.beginPath();
625         fg.arc(this.x, this.y, this.r, 0, 2*Math.PI);
626         fg.fill();
627     }
628 }
629 // ----- DRAW ACCUMULATED DOTS + LINES -----
630 function drawHits() {
631     // CLASSIC STYLE: 3-D - draw a single round dot for every stored offset
632     if (chkClassic.checked) {
633         for (let y = 0; y < screenBottom; y++) {
634             const offsets = hitOffsets[y]; // array of random offsets for this row
635             if (!offsets || offsets.length === 0) continue;
636             // Draw every dot that belongs to this row
637             offsets.forEach(off => {
638                 // `off` is the random horizontal offset (relative to the screen line)
639                 const x = screenX + Math.round(off);
640                 drawRoundDot(x, y, particleSize, 'red');
641             });
642         }
643         // Classic-style is done - exit early
644         return;
645     }
646     // NORMAL MODE: 2-D - draw a simple red line per row
647     for (let y = 0; y < screenBottom; y++) {
648         const idx = hitCount[y];
649         if (idx === 0) continue;
650         const offset = idx * particleSize; // spacing = particleSize
651         bg.strokeStyle = 'red';
652         bg.lineWidth = particleSize;
653         bg.beginPath();
654         bg.moveTo(screenX - 1 + offset, y); // start right from the screen line
655         bg.lineTo(screenX + offset + particleSize, y);
656         bg.stroke();
657     }
658 }
659 function drawRoundDot(x, y, particleSize, colour = 'red') {
660     // In Classic-Style mode the imprint should consist of round dots,
661     // whereas in the regular mode it should be rendered as lines (accumulated dots)
662
663     // x, y - coordinates of the *top-left* corner where you used fillRect
664     // size - the same value you passed as particleSize (width & height)
665     const radius = particleSize / 2; // half the side length
666     const cx = x + radius; // centre X
667     const cy = y + radius; // centre Y
668     bg.fillStyle = colour;
669     bg.beginPath();
670     bg.arc(cx, cy, radius, 0, Math.PI * 2); // full circle
671     bg.fill();
672 }
673 function haltEmission() {
674     /*
675     * Called whenever the imprint reaches the 10 000 hits limit (or any other
676     * condition that forces the emitter to stop).
677     * It puts the UI back into the "ready-to-start" state.
678     */
679     // Stop generating new particles
680     emitting = false;
681     hitLimitReached = true; // remember why we stopped
682     // Update the button text
683     toggleBtn.textContent = 'Start';
684     // Disable the button so the user cannot click it again
685     // until they deliberately reset the simulation.
686     toggleBtn.disabled = true;
687     resetBtn.disabled = false;
688 }
689 // ----- MAIN ANIMATION LOOP -----
690 let particles = [];
691 function animate() {
692     fg.clearRect(0,0,W,H); // Clear only the foreground (dynamic) canvas
693     drawHits(); // Draw accumulated dots + lines
694     particles.forEach(p => p.update()); // Update and draw active particles
695     particles = particles.filter(p => p.alive); // Remove dead particles
696     // If we are in the "emitting" state and no hitLimitReached, spawn a new particle each frame
697     if (emitting && !hitLimitReached) particles.push(new Particle());
698     requestAnimationFrame(animate);
699 }
700 // ----- INITIALISE -----
701 drawBackground();
702 // ----- Draw the tiled watermark -----

```

```

703   for (let r = 0; r < rows; r++) {
704     for (let c = 0; c < cols; c++) {
705       // Compute the centre point for this tile
706       const x = c * colSpacing + colSpacing / 2;
707       const y = r * lineSpacing + lineSpacing / 2;
708       wm.save(); // isolate transformations per tile
709       wm.translate(x, y); // move origin to tile centre
710       wm.rotate(angleWm); // rotate the text
711       wm.textBaseline = 'bottom'; // split the text in 2 lines
712       wm.fillText('copyright © 2025 Ion Vlad', 0, 0); // and draw the watermark
713       wm.textBaseline = 'top';
714       wm.fillText('contact@ionvlad.com', 0, 0);
715       wm.restore(); // revert to default state
    }
  }
  info.setAttribute('style', 'white-space: pre-line;');
  info.textContent = '';
  animate();
  //--- Mouse handling – move the lens and redraw its contents ----
  function onMouseMove(e) {
    const rect = bgCanvas.getBoundingClientRect();
    const mouseX = e.clientX - rect.left;
    const mouseY = e.clientY - rect.top + canvasTop;
    if (mouseX >= screenX - lensSize){
      // if mouse close to screen line, then show magnifier
      // and position the lens so its centre follows the pointer
      lens.style.left = `${mouseX - lensSize/2}px`;
      lens.style.top = `${mouseY - lensSize/2}px`;
      lens.style.display = 'block';
    } else {
      lens.style.display = 'none';
    }
    ctxLens.clearRect(0,0,lensSize,lensSize); // Clear previous frame
    // Clip to a circle
    ctxLens.save();
    ctxLens.beginPath();
    ctxLens.arc(lensSize/2, lensSize/2, lensSize/2, 0, Math.PI*2);
    ctxLens.clip();
    ctxLens.drawImage(
      bgCanvas, // source canvas
      mouseX - srcRadius, // sx
      mouseY - srcRadius - canvasTop, // sy
      srcRadius*2, // sWidth
      srcRadius*2, // sHeight
      0, 0, // dx, dy (top-left of lens canvas)
      lensSize, lensSize // dWidth, dHeight (scaled up)
    );
  }
  /* Hide the lens when the pointer leaves the canvas */
  function onMouseLeave() {
    lens.style.display = 'none';
  }
  /* Attach listeners */
  fgCanvas.addEventListener('mousemove', onMouseMove);
  fgCanvas.addEventListener('mouseleave', onMouseLeave);
</script>
</body>
</html>

```